

Object-Oriented Programming (OOP) in Python

Introduction to Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm that organizes code using **objects** and **classes**. It helps developers create reusable, maintainable, and scalable applications by modeling real-world entities as software objects.

Python is a multi-paradigm language that fully supports Object-Oriented Programming.

Why Use OOP?

- Code reusability
 - Better organization of programs
 - Easier maintenance
 - Improved scalability
 - Real-world modeling
 - Enhanced security through encapsulation
-

Basic Terminology

1. Class

A class is a blueprint or template used to create objects.

Example

```
class Student:  
    pass
```

Here, Student is a class.

2. Object

An object is an instance of a class.

Example

```
class Student:  
    pass
```

```
s1 = Student()  
s2 = Student()
```

Here:

- s1 and s2 are objects of the Student class.
-

Creating a Class and Object

Example

```
class Student:
```

```
    def display(self):  
        print("Welcome to Python OOP")
```

```
s1 = Student()  
s1.display()
```

Output

Welcome to Python OOP

Constructor (init)

A constructor is a special method that automatically executes when an object is created.

Syntax

```
def __init__(self):  
    # initialization code
```

Example

```
class Student:
```

```
    def __init__(self):  
        print("Object Created")
```

```
s1 = Student()
```

Output

Object Created

The self Keyword

self refers to the current object.

Example

```
class Student:
```

```
def show(self):  
    print("This is self keyword")
```

```
s1 = Student()  
s1.show()
```

Python automatically passes the object as the first argument.

Instance Variables

Instance variables belong to individual objects.

Example

```
class Student:
```

```
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

```
s1 = Student("Rahul", 20)
```

```
print(s1.name)  
print(s1.age)
```

Output

```
Rahul  
20
```

Instance Methods

Methods that work with instance variables.

Example

```
class Student:
```

```
    def __init__(self, name):  
        self.name = name
```

```
    def display(self):  
        print("Student Name:", self.name)
```

```
s1 = Student("Amit")  
s1.display()
```

Output

Student Name: Amit

Class Variables

Class variables are shared among all objects.

Example

class Student:

```
    college = "ABC College"
```

```
    def __init__(self, name):  
        self.name = name
```

```
s1 = Student("Rahul")
```

```
s2 = Student("Amit")
```

```
print(s1.college)
```

```
print(s2.college)
```

Output

ABC College

ABC College

Difference Between Instance and Class Variables

Instance Variable	Class Variable
Belongs to object	Belongs to class
Separate copy for each object	Shared by all objects
Created using self	Created inside class directly

Four Pillars of OOP

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

1. Encapsulation

Encapsulation means binding data and methods together into a single unit (class).

It protects data from unauthorized access.

Example

```
class BankAccount:
```

```
    def __init__(self):  
        self.balance = 1000
```

```
    def show_balance(self):  
        print(self.balance)
```

```
acc = BankAccount()  
acc.show_balance()
```

Public, Protected, and Private Members

Public Members

Accessible from anywhere.

```
class Student:
```

```
    def __init__(self):  
        self.name = "Rahul"
```

Protected Members

Represented using a single underscore _.

```
class Student:
```

```
    def __init__(self):  
        self._name = "Rahul"
```

Convention suggests not accessing it directly outside the class.

Private Members

Represented using double underscore __.

```
class Student:
```

```
    def __init__(self):  
        self.__salary = 50000
```

Cannot be directly accessed outside the class.

Example

```
class Employee:
```

```
    def __init__(self):  
        self.__salary = 50000
```

```
emp = Employee()
```

```
# print(emp.__salary)
```

This will generate an error.

2. Inheritance

Inheritance allows one class to acquire properties and methods of another class.

Benefits

- Code reuse
 - Easy maintenance
 - Reduced redundancy
-

Types of Inheritance in Python

1. Single Inheritance
 2. Multiple Inheritance
 3. Multilevel Inheritance
 4. Hierarchical Inheritance
 5. Hybrid Inheritance
-

Single Inheritance

```
class Parent:
```

```
def display(self):
    print("Parent Class")

class Child(Parent):
    pass

obj = Child()
obj.display()
```

Output

Parent Class

Multiple Inheritance

```
class Father:

    def skill1(self):
        print("Driving")

class Mother:

    def skill2(self):
        print("Cooking")

class Child(Father, Mother):
    pass

c = Child()

c.skill1()
c.skill2()
```

Multilevel Inheritance

```
class Grandfather:

    def property(self):
        print("Land")

class Father(Grandfather):
    pass

class Child(Father):
```

```
pass
```

```
obj = Child()  
obj.property()
```

Hierarchical Inheritance

```
class Parent:
```

```
    def display(self):  
        print("Parent Class")
```

```
class Child1(Parent):  
    pass
```

```
class Child2(Parent):  
    pass
```

Method Overriding

When a child class provides its own implementation of a parent class method.

Example

```
class Animal:
```

```
    def sound(self):  
        print("Animal Sound")
```

```
class Dog(Animal):
```

```
    def sound(self):  
        print("Bark")
```

```
d = Dog()  
d.sound()
```

Output

Bark

super() Function

Used to access parent class methods and constructors.

Example

class Parent:

```
def __init__(self):  
    print("Parent Constructor")
```

class Child(Parent):

```
def __init__(self):  
    super().__init__()  
    print("Child Constructor")
```

c = Child()

Output

Parent Constructor

Child Constructor

3. Polymorphism

Polymorphism means "many forms".

The same method behaves differently for different objects.

Example 1: Method Overriding

class Bird:

```
def sound(self):  
    print("Bird Sound")
```

class Sparrow(Bird):

```
def sound(self):  
    print("Chirp")
```

class Crow(Bird):

```
def sound(self):  
    print("Caw")
```

birds = [Sparrow(), Crow()]

```
for bird in birds:  
    bird.sound()
```

Output

```
Chirp  
Caw
```

Example 2: Operator Overloading

```
print(5 + 10)  
  
print("Hello " + "World")
```

Output

```
15  
Hello World
```

The + operator behaves differently for integers and strings.

4. Abstraction

Abstraction means hiding implementation details and showing only essential functionality. Python provides abstraction using the abc module.

Abstract Class

```
from abc import ABC, abstractmethod  
  
class Vehicle(ABC):  
  
    @abstractmethod  
    def start(self):  
        pass
```

Implementing Abstract Class

```
from abc import ABC, abstractmethod  
  
class Vehicle(ABC):  
  
    @abstractmethod  
    def start(self):
```

```
        pass

class Car(Vehicle):

    def start(self):
        print("Car Started")

c = Car()
c.start()
```

Output

Car Started

Class Methods

Class methods work with class variables.

Example

```
class Student:

    college = "ABC College"

    @classmethod
    def get_college(cls):
        return cls.college

print(Student.get_college())
```

Static Methods

Static methods do not depend on class variables or instance variables.

Example

```
class Math:

    @staticmethod
    def add(a, b):
        return a + b

print(Math.add(5, 10))
```

Output

15

Magic (Dunder) Methods

Special methods surrounded by double underscores.

Examples:

```
__init__()  
__str__()  
__len__()  
__add__()
```

Example

```
class Student:
```

```
    def __init__(self, name):  
        self.name = name
```

```
    def __str__(self):  
        return self.name
```

```
s = Student("Rahul")
```

```
print(s)
```

Output

```
Rahul
```

Real-World Example: Student Management System

```
class Student:
```

```
    college = "InternStudio Academy"
```

```
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

```
    def display(self):  
        print("Name:", self.name)  
        print("Age:", self.age)
```

```
s1 = Student("Rahul", 20)
```

```
s2 = Student("Amit", 22)
```

```
s1.display()  
print()
```

```
s2.display()
```

Advantages of OOP

1. Code Reusability
 2. Modularity
 3. Data Security
 4. Easy Debugging
 5. Scalability
 6. Better Code Structure
 7. Real-World Modeling
 8. Faster Development
-

Disadvantages of OOP

1. Requires More Planning
 2. Slightly Slower Execution
 3. Can Become Complex for Small Projects
 4. More Memory Consumption
-

Sample Programs:

1. Student Class Program

```
class Student:
```

```
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

```
    def display(self):  
        print("Name:", self.name)  
        print("Age:", self.age)
```

```
s1 = Student("Rahul", 20)
```

```
s1.display()
```

Output

Name: Rahul

Age: 20

2. Rectangle Area Calculator

```
class Rectangle:
```

```
    def __init__(self, length, width):  
        self.length = length  
        self.width = width
```

```
    def area(self):  
        return self.length * self.width
```

```
r1 = Rectangle(10, 5)
```

```
print("Area:", r1.area())
```

Output

Area: 50

3. Bank Account System

```
class BankAccount:
```

```
    def __init__(self, balance):  
        self.balance = balance
```

```
    def deposit(self, amount):  
        self.balance += amount
```

```
    def withdraw(self, amount):  
        self.balance -= amount
```

```
    def show_balance(self):  
        print("Balance:", self.balance)
```

```
acc = BankAccount(5000)
```

```
acc.deposit(1000)
```

```
acc.withdraw(500)
```

```
acc.show_balance()
```

Output

Balance: 5500

4. Single Inheritance Example

```
class Animal:
```

```
    def eat(self):
```

```
        print("Animal is eating")
```

```
class Dog(Animal):
```

```
    def bark(self):
```

```
        print("Dog is barking")
```

```
d = Dog()
```

```
d.eat()
```

```
d.bark()
```

Output

Animal is eating

Dog is barking

5. Multilevel Inheritance Example

```
class Grandparent:
```

```
    def house(self):
```

```
        print("Owns a House")
```

```
class Parent(Grandparent):
```

```
    def car(self):
```

```
        print("Owns a Car")
```

```
class Child(Parent):

    def bike(self):
        print("Owns a Bike")

c = Child()

c.house()
c.car()
c.bike()
```

6. Method Overriding Example

```
class Animal:

    def sound(self):
        print("Animal Sound")

class Dog(Animal):

    def sound(self):
        print("Bark")

d = Dog()

d.sound()
```

Output

Bark

7. Employee Salary System

```
class Employee:

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def show_details(self):
        print("Name:", self.name)
        print("Salary:", self.salary)

e1 = Employee("Amit", 45000)
```



```
e1.show_details()
```

8. Encapsulation Example

```
class Employee:

    def __init__(self):
        self.__salary = 50000

    def get_salary(self):
        return self.__salary

emp = Employee()

print(emp.get_salary())
```

Output

```
50000
```

9. Class Variable Example

```
class Student:

    college = "InternStudio Academy"

    def __init__(self, name):
        self.name = name

s1 = Student("Rahul")
s2 = Student("Amit")

print(s1.college)
print(s2.college)
```

Output

```
InternStudio Academy
InternStudio Academy
```

10. Class Method Example

```
class Student:

    college = "InternStudio Academy"

    @classmethod
    def get_college(cls):
        return cls.college

print(Student.get_college())
```

Output

InternStudio Academy

11. Static Method Example

```
class Calculator:

    @staticmethod
    def add(a, b):
        return a + b

print(Calculator.add(10, 20))
```

Output

30

12. Abstract Class Example

```
from abc import ABC, abstractmethod

class Vehicle(ABC):

    @abstractmethod
    def start(self):
        pass

class Car(Vehicle):

    def start(self):
        print("Car Started")

c = Car()
```

```
c.start()
```

Output

Car Started

13. Library Management System

```
class Book:
```

```
    def __init__(self, title, author):  
        self.title = title  
        self.author = author
```

```
    def display(self):  
        print("Title:", self.title)  
        print("Author:", self.author)
```

```
b1 = Book("Python Programming", "John Smith")
```

```
b1.display()
```

14. Shopping Cart Program

```
class Product:
```

```
    def __init__(self, name, price):  
        self.name = name  
        self.price = price
```

```
    def display(self):  
        print(self.name, "-", self.price)
```

```
p1 = Product("Laptop", 50000)
```

```
p2 = Product("Mouse", 800)
```

```
p1.display()
```

```
p2.display()
```

15. Simple ATM System

```
class ATM:

    def __init__(self, balance):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print("Deposited:", amount)

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print("Withdrawn:", amount)
        else:
            print("Insufficient Balance")

    def show_balance(self):
        print("Balance:", self.balance)

atm = ATM(10000)

atm.deposit(2000)
atm.withdraw(3000)
atm.show_balance()
```

16. Student Result Management System

```
class Student:

    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def calculate_grade(self):

        if self.marks >= 90:
            return "A"

        elif self.marks >= 75:
            return "B"

        elif self.marks >= 50:
            return "C"
```

```
    else:
        return "Fail"

    def display(self):
        print("Name:", self.name)
        print("Marks:", self.marks)
        print("Grade:", self.calculate_grade())

s1 = Student("Rahul", 85)

s1.display()
```

Output

```
Name: Rahul
Marks: 85
Grade: B
```

Summary

Object-Oriented Programming is one of the most important concepts in Python. It helps developers build structured and reusable applications by organizing code into classes and objects. The four pillars of OOP—**Encapsulation, Inheritance, Polymorphism, and Abstraction**—form the foundation of modern software development. Understanding OOP is essential for creating large-scale applications, web applications, desktop software, automation tools, and enterprise systems in Python.